ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
DATA ENGINEERING (CO5173)
NHÓM 16

ỨNG DỤNG XỬ LÝ MICRO-BATCH TRONG GIÁM SÁT VÀ QUẢN LÝ NÔNG NGHIỆP

GVHD: PGS.TS. VÕ THỊ NGỌC CHÂU

Sinh viên:	Đoàn Trương Thanh Tịnh	2570761
	Nguyễn Hà Tiến Đạt	2670265
	Lê Duy Khang	2670301
	Hoàng Chí Nhân	2670343

TP. Hồ Chí Minh, Ngày 16 tháng 10 năm 2026

LỜI CAM KẾT

Nhóm xin cam đoan đây là công trình của nhóm, được thực hiện dưới sự hướng dẫn của PGS.TS. Võ Thị Ngọc Châu. Mọi tài liệu và công cụ của bên thứ ba được sử dụng đều ghi rõ nguồn gốc.

Chữ kí GVHD

Trân trọng,

Nhóm 16

TÓM LƯỢC

[Tóm lược báo cáo: bối cảnh, giải pháp micro-batch, kết quả đánh giá chính.]

Từ khóa: Micro-batch, Spark Structured Streaming, Kafka, Nông nghiệp, Data Engineering.

Mục lục

Danh mục từ viết tắt **vii**

Chương 1 GIỚI THIỆU TỔNG QUAN **1**

1.1 Bối cảnh đề tài . . . 1
1.2 Mục tiêu của đề tài . . . 1
1.3 Phạm vi của đề tài . . . 1
1.4 Phân công công việc . . . 1
1.5 Bố cục báo cáo . . . 1

Chương 2 CƠ SỞ LÝ THUYẾT **2**

2.1 Thách thức của xử lý luồng thuần túy . . . 2
2.1.1 Khôi phục trạng thái đang xử lý . . . 2
2.1.2 Ghi trùng khi ghi dở dang . . . 2
2.1.3 Độ phức tạp trạng thái ở mức dưới một giây . . . 3
2.2 Khái niệm xử lý micro-batch . . . 3
2.3 Cơ chế hoạt động . . . 3
2.3.1 Kiến trúc pipeline . . . 3
2.3.2 Vòng đời xử lý của một micro-batch . . . 4
2.3.3 Chu kỳ trigger và ví dụ dữ liệu nông nghiệp . . . 4
2.3.4 Độ tin cậy: phục hồi sau lỗi và dữ liệu đến muộn . . . 4
2.4 Công nghệ hiện thực . . . 5
2.4.1 Apache Kafka làm nguồn dữ liệu . . . 5
2.4.2 Spark Structured Streaming . . . 5
2.4.3 Checkpoint . . . 6
2.4.4 Ví dụ mã nguồn . . . 6
2.5 So sánh, trường hợp sử dụng và đánh đổi . . . 7
2.5.1 So sánh ba mô hình xử lý . . . 7
2.5.2 Trường hợp sử dụng . . . 7
2.5.3 Được và mất . . . 8
2.5.4 Liên hệ với đề tài . . . 8

Chương 3 PHÂN TÍCH YÊU CẦU VÀ DỮ LIỆU **9**

3.1 Lĩnh vực ứng dụng: giám sát nông nghiệp . . . 10
3.2 Các bên liên quan . . . 10
3.3 Yêu cầu nghiệp vụ . . . 10

3.4 Nhu cầu sử dụng dữ liệu . . . 11
3.5 Nguồn dữ liệu và đặc trưng dữ liệu . . . 11
3.5.1 Nguồn dữ liệu . . . 11
3.5.2 Đặc trưng dữ liệu . . . 11
3.6 Ánh xạ yêu cầu nghiệp vụ sang yêu cầu xử lý dữ liệu . . . 11
Chương 4 THIẾT KẾ GIẢI PHÁP VÀ HIỆN THỰC ỨNG DỤNG 12
4.1 Kiến trúc tổng thể . . . 13
4.2 Công nghệ quản lý dữ liệu . . . 13
4.3 Công nghệ xử lý dữ liệu . . . 13
4.4 Thiết kế mô hình dữ liệu . . . 13
4.5 Pipeline micro-batch theo từng yêu cầu nghiệp vụ . . . 13
4.5.1 BR01 – Tập trung hóa giám sát nông nghiệp . . . 13
4.5.2 BR02 – Nâng cao nhận biết điều kiện đồng ruộng . . . 13
4.5.3 BR03 – Duy trì tính liên tục của giám sát . . . 13
4.5.4 BR04 – Thiết lập dữ liệu đáng tin cậy . . . 13
4.5.5 BR05 – Hỗ trợ quản lý dựa trên dữ liệu . . . 13
4.5.6 BR06 – Đánh giá lịch sử và lập kế hoạch . . . 13
4.5.7 BR07 – Làm cho thông tin dễ tiếp cận . . . 13
4.5.8 BR08 – Hỗ trợ ứng phó sớm với rủi ro . . . 13
4.6 Giải pháp thay thế để so sánh . . . 13
4.7 Hiện thực ứng dụng . . . 13
4.7.1 Mô phỏng và thu thập dữ liệu . . . 13
4.7.2 Giao diện người dùng . . . 13
4.7.3 Triển khai và cấu hình . . . 13
Chương 5 ĐÁNH GIÁ 14
5.1 Thiết lập thực nghiệm . . . 14
5.2 Tính đúng đắn của dữ liệu . . . 14
5.3 Hiệu năng của giải pháp . . . 14
5.4 Hiệu quả hỗ trợ khai thác dữ liệu qua ứng dụng . . . 14
5.5 So sánh với giải pháp thay thế . . . 14
5.6 Thảo luận . . . 14
Chương 6 KẾT LUẬN 15
6.1 Kết quả đạt được . . . 15
6.2 Hạn chế . . . 15
6.3 Hướng phát triển . . . 15
Tài liệu tham khảo 16

Danh sách hình ảnh

2.1 Rời rạc hóa luồng dữ liệu thành các micro-batch 3
2.2 Kiến trúc pipeline micro-batch từ cảm biến đến bảng điều khiển 4

Danh sách bảng

2.1 Ví dụ hai micro-batch 5 giây của một cảm biến độ ẩm đất 4
2.2 So sánh xử lý theo lô, micro-batch và xử lý luồng 7
2.3 Lợi ích và chi phí của micro-batch 8

3.1 Danh sách yêu cầu nghiệp vụ (Business Requirements) 10

Danh mục từ viết tắt

Viết tắt	Tiếng Anh	Tiếng Việt
BR	Business Requirement	Yêu cầu nghiệp vụ
ETL	Extract – Transform – Load	Trích xuất – biến đổi – nạp dữ liệu
IoT	Internet of Things	Internet vạn vật
LoRaWAN	Long Range Wide Area Network	Mạng diện rộng tầm xa

CHƯƠNG 1

GIỚI THIỆU TỔNG QUAN

1.1 Bối cảnh đề tài

1.2 Mục tiêu của đề tài

1.3 Phạm vi của đề tài

1.4 Phân công công việc

1.5 Bố cục báo cáo

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

Chương này trình bày nền tảng lý thuyết của kỹ thuật xử lý micro-batch, là kỹ thuật xử lý dữ liệu mà đề tài áp dụng cho lĩnh vực nông nghiệp. Mục 2.1 nêu các thách thức của xử lý luồng thuần túy, là động lực dẫn tới micro-batch. Mục 2.2 định nghĩa micro-batch. Mục 2.3 giải thích cơ chế hoạt động, từ kiến trúc, vòng đời một lô đến cách xử lý lỗi và dữ liệu đến muộn. Mục 2.4 giới thiệu bộ công nghệ Apache Kafka và Spark Structured Streaming. Mục 2.5 so sánh micro-batch với xử lý theo lô và xử lý luồng, cùng các đánh đổi.

2.1 Thách thức của xử lý luồng thuần túy

Trong xử lý luồng thuần túy (pure stream processing), mỗi sự kiện được xử lý ngay khi đến. Cách làm này cho độ trễ thấp, nhưng khi áp dụng cho dữ liệu cảm biến nông nghiệp thì nảy sinh ba khó khăn chính.

2.1.1 Khôi phục trạng thái đang xử lý

Khi một nút xử lý gặp sự cố giữa chừng, hệ thống phải dựng lại trạng thái phức tạp của hàng nghìn sự kiện chưa được đệm. Ví dụ, một nút bị lỗi khi đang nhận đợt dữ liệu dồn kéo dài 4 giờ từ cổng LoRaWAN sẽ rơi vào vòng lặp khởi động lại liên tục vì phải khôi phục dữ liệu đất chưa được đệm.

2.1.2 Ghi trùng khi ghi dở dang

Xử lý từng sự kiện không có ranh giới thực thi rõ ràng. Khi mạng đứt giữa chừng, việc tránh ghi trùng đòi hỏi cơ chế khóa phân tán tốn kém. Ví dụ, kết nối LoRaWAN đứt giữa lúc đồng bộ dữ liệu tưới khiến gói tin gửi lại bị cộng hai lần vào nhật ký lượng nước.

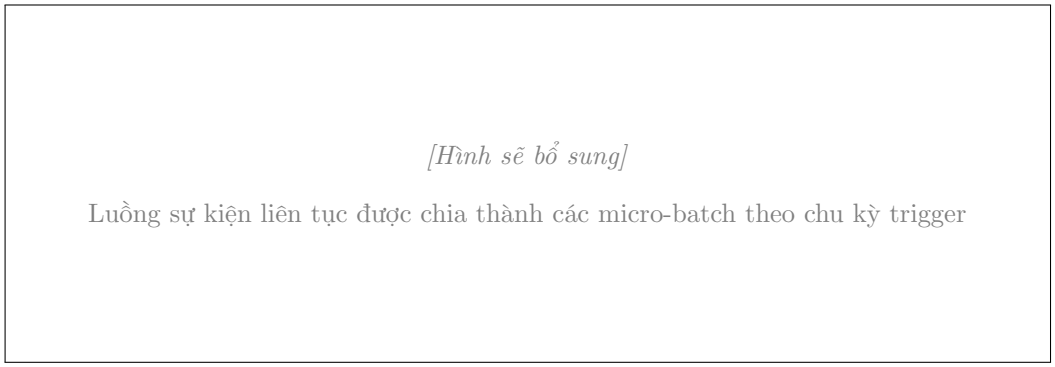
2.1.3 Độ phức tạp trạng thái ở mức dưới một giây

Xử lý luồng liên tục cần cơ chế watermark cho dữ liệu đến sai thứ tự và tiêu tốn nhiều bộ nhớ, làm tăng chi phí cho những tác vụ không cần độ trễ dưới một giây. Ví dụ, chạy vòng đánh giá dưới một giây trên nhiệt độ đất vốn biến thiên chậm là lãng phí, vì chu kỳ cập nhật 30 giây đã đáp ứng nhu cầu canh tác.

2.2 Khái niệm xử lý micro-batch

Micro-batch là mô hình xử lý lai: chia một luồng dữ liệu liên tục, không giới hạn thành chuỗi các khối dữ liệu nhỏ, hữu hạn (micro-batch), mỗi khối được thực thi sau một khoảng thời gian ngắn. Kleppmann mô tả ý tưởng này là "chia luồng thành các khối nhỏ và xử lý mỗi khối như một tiến trình xử lý theo lô thu nhỏ" [1].

Nhờ ranh giới rõ ràng của từng khối, mỗi lô có thể được ghi nhận, thử lại và cam kết (commit) như một đơn vị độc lập. Mô hình kết hợp độ trễ thấp của xử lý luồng với thông lượng cao và khả năng chịu lỗi của xử lý theo lô [2].

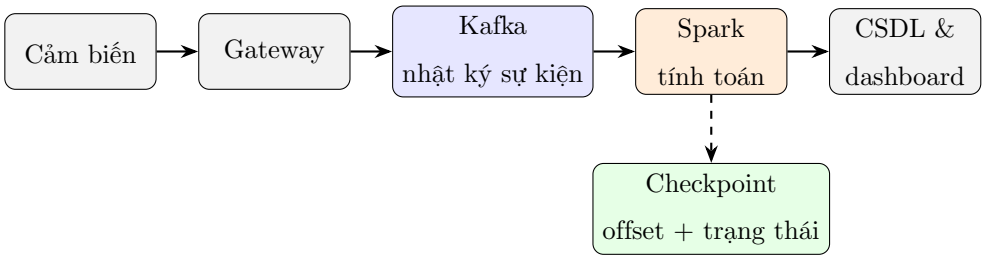


Hình 2.1: Rời rạc hóa luồng dữ liệu thành các micro-batch

2.3 Cơ chế hoạt động

2.3.1 Kiến trúc pipeline

Một pipeline micro-batch điển hình cho dữ liệu cảm biến gồm các thành phần nối tiếp trong Hình 2.2. Dữ liệu từ cảm biến qua cổng kết nối (gateway) vào nhật ký sự kiện Kafka. Kafka tiếp tục nhận sự kiện trong khi Spark đang tính toán, nhờ đó khâu thu thập và khâu xử lý tách rời nhau. Kết quả được ghi vào cơ sở dữ liệu phục vụ bảng điều khiển. Tiến độ xử lý và trạng thái được lưu bền vững trong checkpoint.



Hình 2.2: Kiến trúc pipeline micro-batch từ cảm biến đến bảng điều khiển

2.3.2 Vòng đời xử lý của một micro-batch

Mỗi micro-batch trải qua sáu bước:

1. **Trigger**: bắt đầu lượt chạy tiếp theo.
2. **Lập kế hoạch và ghi offset**: chọn các bản ghi mới và ghi lại phạm vi offset sẽ đọc.
3. **Đọc và biến đổi**: áp dụng truy vấn lên dữ liệu của lô.
4. **Cập nhật trạng thái**: dành cho các truy vấn có trạng thái (cửa sổ, tổng hợp, khử trùng lặp).
5. **Ghi kết quả**: gửi đầu ra tới nơi lưu (sink).
6. **Cam kết lô**: ghi nhận lô đã hoàn tất.

2.3.3 Chu kỳ trigger và ví dụ dữ liệu nông nghiệp

Chu kỳ trigger quyết định độ chi tiết của lô. Giả sử một cảm biến độ ẩm đất gửi số đo lên Kafka, và trigger chạy mỗi 5 giây (Bảng 2.1). Mỗi lô tính một giá trị tổng hợp rồi lưu kết quả, hiển thị trên bảng điều khiển và đưa vào các quy tắc cảnh báo.

Bảng 2.1: Ví dụ hai micro-batch 5 giây của một cảm biến độ ẩm đất

Lô	**Trigger**	**Số đo đến Kafka (độ ẩm %)**	**Trung bình**
1	08:00:05	22 (08:00:01), 24 (08:00:03), 26 (08:00:04)	24%
2	08:00:10	28 (08:00:07), 30 (08:00:09)	29%

2.3.4 Độ tin cậy: phục hồi sau lỗi và dữ liệu đến muộn

Phục hồi sau lỗi. Checkpoint bền vững lưu offset và trạng thái truy vấn. Khi khởi động lại, hệ thống phát lại và hoàn tất lô chưa được cam kết, nên không mất và không bỏ sót dữ liệu.

Dữ liệu đến muộn. Cần phân biệt thời gian sự kiện (event time) với thời gian đến (arrival time). Một số đo có thể được tạo lúc 08:00:03 nhưng đến hệ thống sau đó vì mất kết nối. Hệ thống giữ trạng thái cửa sổ để nhận các số đo muộn này. *Watermark* đi theo tiến độ của thời gian sự kiện; nó không phải bộ đếm chờ theo đồng hồ thực. Khi watermark tiến lên, trạng thái của các cửa sổ đủ điều kiện được dọn, giữ bộ nhớ ở mức giới hạn.

Điều kiện exactly-once. Kết quả "đúng một lần" (exactly-once) cần đồng thời ba điều kiện: nguồn có thể phát lại (replayable), engine có checkpoint, và nơi ghi an toàn khi thử lại (idempotent hoặc transactional). Nếu chỉ dùng sink Kafka thông thường thì chỉ đạt at-least-once.

2.4 Công nghệ hiện thực

Bộ ba công nghệ được dùng để hiện thực micro-batch gồm Apache Kafka làm nguồn, Spark Structured Streaming làm engine xử lý và checkpoint bảo đảm độ tin cậy.

2.4.1 Apache Kafka làm nguồn dữ liệu

Kafka là nhật ký sự kiện bền vững và có thể phát lại [3]. Dữ liệu được tổ chức theo *topic*, mỗi topic chia thành các *partition*; mỗi thông điệp trong partition có một *offset* tăng dần. Thông điệp được giữ trong một thời gian lưu trữ (retention) kể cả sau khi đã đọc, nên Spark có thể đọc lại bất kỳ khoảng offset nào. Kafka cũng tách bên sinh dữ liệu khỏi bên xử lý. Một partition Kafka tương ứng một tác vụ (task) Spark.

Ở đầu mỗi lô, Spark hỏi Kafka offset mới nhất của từng partition, rồi xác định khoảng offset cần đọc (ví dụ partition 0: 1000–1500, partition 1: 980–1420). Khoảng offset này chính là micro-batch. Các partition được đọc song song, và offset được theo dõi bằng checkpoint của Spark chứ không dùng consumer group. Tùy chọn `maxOffsetsPerTrigger` giới hạn số bản ghi mỗi lô, bảo vệ tác vụ khỏi các đợt tăng đột biến.

2.4.2 Spark Structured Streaming

Spark Structured Streaming coi luồng là một *bảng không giới hạn*: mỗi bản ghi mới là một dòng được nối vào bảng đầu vào. Ở mỗi trigger, Spark chạy truy vấn chỉ trên các dòng mới và cập nhật kết quả [4]. Nhờ đó cùng một API DataFrame/SQL dùng được cho cả xử lý theo lô và xử lý luồng. Ba khái niệm cấu hình chính:

- **Trigger** (khi nào chạy): `processingTime` chạy theo chu kỳ cố định, ví dụ mỗi 10 giây; mặc định là các lô nối tiếp nhau; `availableNow` xử lý hết dữ liệu hiện có rồi dừng.

- **Output mode** (ghi gì): *append* chỉ ghi dòng mới, *update* chỉ ghi dòng thay đổi, *complete* ghi lại toàn bộ bảng kết quả.
- **Watermark** (xử lý dữ liệu muộn): ví dụ "chờ tối đa 2 phút rồi chốt cửa sổ"; trạng thái cũ bị xóa nên bộ nhớ không tăng vô hạn.

2.4.3 Checkpoint

Thư mục checkpoint lưu bốn thành phần:

- `offsets/`: nhật ký ghi trước (write-ahead log). *Trước* khi lô N chạy, Spark ghi phạm vi offset sẽ đọc.
- `commits/`: *sau* khi lô N được ghi vào sink, Spark đánh dấu lô đã xong.
- `state/`: kho trạng thái cho cửa sổ, tổng hợp và khử trùng lặp.
- `metadata`: mã định danh duy nhất của truy vấn. Khi đổi lớn logic truy vấn, cần dùng vị trí checkpoint mới.

Quy trình phục hồi như sau: ghi offset vào `offsets/`, xử lý và ghi vào sink, rồi ghi `commits/`. Nếu hệ thống sập khi lô N đã có trong `offsets/` nhưng chưa có trong `commits/`, lô N được phát lại với *cùng* khoảng offset. Để kết quả đúng một lần, sink phải idempotent hoặc transactional, ví dụ Delta Lake hoặc upsert theo mã lô.

2.4.4 Ví dụ mã nguồn

Đoạn mã dưới đây đọc luồng đơn hàng từ Kafka, tính doanh thu theo sản phẩm trong cửa sổ 1 phút và cập nhật kết quả mỗi 10 giây. Cùng mẫu này sẽ được áp dụng cho số đo cảm biến trong Chương 4.

```
orders = (spark.readStream.format("kafka")
    .option("subscribe", "orders")
    .option("maxOffsetsPerTrigger", 10000)
    .load())

revenue = (orders
    .select(from_json("value", schema).alias("o")).select("o.*")
    .withWatermark("event_time", "2 minutes")
    .groupBy(window("event_time", "1 minute"), "product")
    .agg(sum("amount")))

(revenue.writeStream
    .outputMode("update")
    .trigger(processingTime="10 seconds")
    .option("checkpointLocation", "/tmp/chk/orders")
```

```
16     .start())
```

Đoạn mã 2.1: Kafka → Spark → sink với trigger 10 giây

2.5 So sánh, trường hợp sử dụng và đánh đổi

2.5.1 So sánh ba mô hình xử lý

Bảng 2.2: So sánh xử lý theo lô, micro-batch và xử lý luồng

Tiêu chí	**Xử lý theo lô**	**Micro-batch**	**Xử lý luồng**
Đơn vị xử lý	Khối dữ liệu lớn	Lô nhỏ, thường xuyên	Từng sự kiện liên tục
Độ trễ	Phút đến giờ	Giây đến phút	Mili giây đến giây
Thông lượng	Cao, hiệu quả	Tốt, độ phức tạp vừa phải	Phụ thuộc thiết kế
Độ phức tạp	Đơn giản	Cân bằng độ trễ và hiệu quả	Cao hơn, chi phí vận hành lớn
Phù hợp với	ETL, báo cáo, phân tích lịch sử	Dashboard, ETL tăng dần, phân tích gần thời gian thực	Phát hiện gian lận, cảnh báo, quyết định thời gian thực

2.5.2 Trường hợp sử dụng

Micro-batch thường dùng cho: phân tích gần thời gian thực, xử lý nhật ký, giám sát IoT, phát hiện gian lận, gợi ý và cá nhân hóa, và các pipeline học máy.

2.5.3 Được và mất

Bảng 2.3: Lợi ích và chi phí của micro-batch

Lợi ích	**Chi phí**
Dữ liệu mới trong vài phút với công cụ theo lô quen thuộc	Độ trễ không thể thấp hơn chu kỳ lô
Thử lại dễ dàng, cam kết idempotent và transactional	Cam kết thường xuyên tạo nhiều tệp nhỏ và snapshot, cần gộp tệp (compaction) và dọn snapshot
Rẻ và đơn giản hơn xử lý luồng thực sự	Chu kỳ càng ngắn thì chi phí lập lịch và siêu dữ liệu càng cao

2.5.4 Liên hệ với đề tài

Dữ liệu nông nghiệp như độ ẩm đất hay nhiệt độ biến thiên chậm, kết nối LoRaWAN dễ gián đoạn, và yêu cầu nghiệp vụ thường chấp nhận độ trễ từ vài giây đến vài chục giây. Các đặc điểm này phù hợp với micro-batch: tránh chi phí của xử lý luồng thuần túy mà vẫn bảo đảm tính liên tục và đúng đắn của dữ liệu. Việc áp dụng cụ thể được trình bày ở Chương 4.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ DỮ LIỆU

3.1 Lĩnh vực ứng dụng: giám sát nông nghiệp

3.2 Các bên liên quan

3.3 Yêu cầu nghiệp vụ

Bảng 3.1: Danh sách yêu cầu nghiệp vụ (Business Requirements)

ID	Yêu cầu	Mô tả	Phụ trách
BR01	Tập trung hóa giám sát nông nghiệp	Cung cấp góc nhìn hợp nhất về trang trại, thửa ruộng và thiết bị giám sát để quản lý nhất quán trên nhiều địa điểm.	Đạt
BR02	Nâng cao nhận biết điều kiện đồng ruộng	Cung cấp kịp thời thông tin về môi trường và điều kiện đồng ruộng, giảm phụ thuộc vào đo thủ công.	[...]
BR03	Duy trì tính liên tục của giám sát	Giảm thiểu khoảng trống dữ liệu và bảo toàn thông tin khi thiết bị, kết nối hoặc dịch vụ gián đoạn tạm thời.	Nhân
BR04	Thiết lập dữ liệu nông nghiệp đáng tin cậy	Bảo đảm thông tin nhất quán, tin cậy, truy vết được; giảm rủi ro do số đo không hợp lệ, trùng lặp hoặc gây hiểu nhầm.	[...]
BR05	Hỗ trợ quản lý trang trại dựa trên dữ liệu	Dùng số đo và xu hướng để định hướng tưới tiêu, phân bổ tài nguyên và quản lý thửa ruộng, giảm lãng phí.	Tịnh
BR06	Đánh giá lịch sử và lập kế hoạch	Lưu trữ lịch sử để so sánh giữa các giai đoạn, nhận diện mùa vụ, đánh giá thực hành quản lý và lập kế hoạch.	[...]
BR07	Làm cho thông tin dễ tiếp cận	Cung cấp tóm tắt, xu hướng và so sánh thửa ruộng dễ hiểu cho quản lý và nhà phân tích.	Duy

3.4 Nhu cầu sử dụng dữ liệu

3.5 Nguồn dữ liệu và đặc trưng dữ liệu

3.5.1 Nguồn dữ liệu

3.5.2 Đặc trưng dữ liệu

3.6 Ánh xạ yêu cầu nghiệp vụ sang yêu cầu xử lý dữ liệu

CHƯƠNG 4

THIẾT KẾ GIẢI PHÁP VÀ HIỆN THỰC ỨNG DỤNG

4.1 Kiến trúc tổng thể

4.2 Công nghệ quản lý dữ liệu

4.3 Công nghệ xử lý dữ liệu

4.4 Thiết kế mô hình dữ liệu

4.5 Pipeline micro-batch theo từng yêu cầu nghiệp vụ

4.5.1 BR01 – Tập trung hóa giám sát nông nghiệp

4.5.2 BR02 – Nâng cao nhận biết điều kiện đồng ruộng

4.5.3 BR03 – Duy trì tính liên tục của giám sát

4.5.4 BR04 – Thiết lập dữ liệu đáng tin cậy

4.5.5 BR05 – Hỗ trợ quản lý dựa trên dữ liệu

4.5.6 BR06 – Đánh giá lịch sử và lập kế hoạch

4.5.7 BR07 – Làm cho thông tin dễ tiếp cận

4.5.8 BR08 – Hỗ trợ ứng phó sớm với rủi ro

4.6 Giải pháp thay thế để so sánh

CHƯƠNG 5

ĐÁNH GIÁ

5.1 Thiết lập thực nghiệm

5.2 Tính đúng đắn của dữ liệu

5.3 Hiệu năng của giải pháp

5.4 Hiệu quả hỗ trợ khai thác dữ liệu qua ứng dụng

5.5 So sánh với giải pháp thay thế

5.6 Thảo luận

CHƯƠNG 6

KẾT LUẬN

6.1 Kết quả đạt được

6.2 Hạn chế

6.3 Hướng phát triển

Tài liệu tham khảo

[1] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, 1st. Sebastopol, CA, USA: O'Reilly Media, 2017.

[2] Conduktor, *Micro-batching: Near-real-time stream processing*, Conduktor Glossary, Truy cập: 10/2026. [Online]. Available: `https://www.conduktor.io/glossary/micro-batching-streaming`

[3] Apache Software Foundation, *Apache kafka documentation.* [Online]. Available: `https://kafka.apache.org/documentation/`

[4] Apache Software Foundation, *Structured streaming programming guide*, Apache Spark Documentation. [Online]. Available: `https://spark.apache.org/docs/latest/streaming/`